

It Is Not the Cryptography: A Discarded Exception Dominates Token Validation Cost in a Node.js Service

Jannatul Ferdous Katha^{1,3}, Tasmia Tahmid Prova^{2,3},
Md Kishor Morol³, Tze Hui Liew^{4*}, Dip Nandi^{3,5}

¹Department of CSE, Bangladesh University of Professionals (BUP),
Dhaka, Bangladesh.

²Department of CSE, Bangladesh University of Business and Technology
(BUBT), Dhaka, Bangladesh.

³ELITE Research Lab, Queens, NY, USA.

⁴Faculty of Information Science and Technology, Multimedia University,
Melaka, Malaysia.

⁵Department of Computer Science, American International
University-Bangladesh (AIUB), Dhaka, Bangladesh.

*Corresponding author(s). E-mail(s): thliew@mmu.edu.my;

Contributing authors: ferdous.katha@elitelab.ai; tasmia.prova@elitelab.ai;
kishormorol@ieee.org; dip.nandi@aiub.edu;

Abstract

The per-request cost of JSON Web Token validation is routinely attributed to cryptography. We report that for a widely used Node.js library the attribution is wrong, and that our own earlier attempt to correct it was also wrong. When a shared secret is supplied as a string — which is what the library’s interface invites — `jsonwebtoken` resolves it by attempting to parse it as an *asymmetric* key first, falling back to the symmetric constructor only after that attempt throws. The conversion it falls back to costs $0.583\ \mu\text{s}$ and explains nothing. The discarded failure preceding it costs $18.04\ \mu\text{s}$, 60% of the call and its largest single component. Two independent methods agree: differencing timed conditions across 15 separate processes, and a sampling profiler attributing a median 66.2% of self time to that parse. Holding containerisation constant and varying the runtime, the magnitude is governed by the bundled OpenSSL version rather than by the

JavaScript engine. The cost persists in a deployed service and depends on arrival rate. The avoiding pattern is rare: 0 of 327 classified call sites across 247 repositories pass a pre-parsed key, and among 112 repositories that both reference the pre-parsing interface and call the validator — the most favourable denominator available — only 6 use it. We propose a fix, and report a constraint on it we consider the more important result: the discarded attempt is load-bearing for an existing key-type defence, so the obvious form of the optimisation must not be applied.

Keywords: JSON Web Token, performance measurement, benchmarking methodology, Node.js, OpenSSL, software defect

1 Introduction

Authentication in cloud-native services is commonly described as imposing a per-request cryptographic cost [Zhu et al \(2023\)](#); [Katha et al \(2026a\)](#). The assumption is old and has been tested before, one layer down: Coarfa et al. [Coarfa et al \(2006\)](#) instrumented TLS web servers and found RSA responsible for only a minority of server cost, the remainder falling to symmetric operations, hashing and protocol handling. Our result is that argument repeated at the application layer, and reaching a sharper version of the same conclusion. The reasoning is intuitive: validating a JSON Web Token [Jones et al \(2015\)](#) means verifying a signature, signatures are cryptography, and cryptography is expensive relative to parsing. The conclusion usually drawn is that the cost should be *moved* — to a sidecar proxy, a gateway, or dedicated hardware — because specialised components do cryptographic work well.

We report that the premise is wrong for the most widely used JavaScript implementation we could measure, and that our own earlier correction of it was wrong in a way worth recording. Our own earlier manuscript [Katha et al \(2026a\)](#) established that the signature primitive is a small share of the cost, and attributed

the remainder to key material being converted from a string into a key object on every call. Measuring that conversion directly puts it at $0.583\ \mu\text{s}$: too small to be the explanation for anything.

The actual mechanism is a control-flow accident. Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed. Every validation therefore constructs and discards an OpenSSL parse failure, at $18.04\ \mu\text{s}$ against a total call cost of $29.92\ \mu\text{s}$.

Research questions

RQ1 Where does the per-request cost of token validation actually go?

RQ2 What governs the magnitude of that cost?

RQ3 Does it matter in a deployed service, and how common is the pattern that avoids it?

Contributions

All supported by measurements in the replication package [Katha et al \(2026b\)](#).

1. A decomposition of the validation call, by two independent methods, locating its dominant component in a discarded

exception rather than in cryptography or in key conversion (Section 5).

2. Evidence that the magnitude is governed by the runtime’s bundled OpenSSL version rather than by the JavaScript engine or by containerisation, using a runtime pair that separates the two (Section 6).
3. A deployed-service measurement across seven arrival rates, and a survey establishing that the avoiding pattern is rare in public code (Section 7).
4. A fix, and a constraint on it: the discarded attempt is load-bearing for an existing key-type defence, so the natural form of the optimisation is unsafe (Section 8).

2 Background and related work

What a validation call does.

A JSON Web Token carries a header, a payload and a signature over the first two [Jones et al \(2015\)](#). Validating one means decoding two segments, resolving key material into a form the cryptographic library accepts, and verifying the signature. The middle step — key resolution — is the subject of this paper, and it is the step most often left implicit when validation cost is discussed.

Where the cost is usually attributed.

Published measurement of layered enforcement in microservices concentrates on transport and proxy layers. Zhu et al. [Zhu et al \(2023\)](#) decompose service-mesh sidecar overhead and show that the dominant contributors depend on configuration and workload rather than being a fixed tax; we reach an analogous conclusion one layer up, for application-level validation. Application-level token validation is itself thinly

evidenced, and is commonly reported as a difference between scenario averages, which attributes to validation every difference between two runs. Our own earlier study did exactly that [Katha et al \(2026a\)](#). We were unable to locate a study that measures the key-resolution step directly; we state that as an absence we searched for rather than as a claim about the whole literature.

Cryptographic cost has been over-attributed before.

Coarfa et al. [Coarfa et al \(2006\)](#) remains the clearest precedent: measuring TLS web servers rather than primitives, they found the public-key operation was a minority of server cost. Work on cryptographic library performance since has largely stayed at the level of the primitive or the protocol exchange [Naser et al \(2020\)](#); [Sikeridis et al \(2020\)](#), with Tuveri and Brumley [Tuveri and Brumley \(2019\)](#) a notable exception in examining how a library loads and dispatches implementations rather than how fast those implementations run. Our result concerns neither: it is the cost of getting key material *into* the library.

Version and environment variation.

That software performance changes across versions in ways maintainers do not anticipate is established: Kaltenacker et al. [Kaltenacker et al \(2023\)](#) track performance across configurable systems’ release histories, and Chen and Shang [Chen and Shang \(2017\)](#) study the code changes that introduce regressions. Laaber and Leitner [Laaber and Leitner \(2018\)](#) assess whether microbenchmark suites detect such changes at all. Container images make this concrete by pinning a dependency set implicitly: Cito et al. [Cito et al \(2017\)](#) characterise how

the ecosystem uses them, and Malka et al. [Malka et al \(2026\)](#) show that an image tag does not guarantee a reproducible build. Section 6 is an instance with a measured cost attached.

Measurement protocol is the thing at stake.

The methodological core of this paper is not new. Georges et al. [Georges et al \(2007\)](#) established that a single execution of a managed-runtime benchmark is not a measurement, and that intervals must be computed across executions rather than within one. Mytkowicz et al. [Mytkowicz et al \(2009\)](#) showed that environmental details unrelated to the code under study can produce effects larger than the one being measured, without any obvious mistake. Traini et al. [Traini et al \(2023\)](#) found that steady state is frequently not reached at all and that developers misjudge when warmup ends. Our contribution is not to these methods but is an instance of what they warn about: both errors we correct in Section 3 have the structure Mytkowicz et al. describe, a second variable moving unrecorded alongside the one under study.

Environment as a confounder.

Laaber et al. [Laaber et al \(2019\)](#) quantify how far cloud environments degrade microbenchmark reliability, and Leitner and Cito [Leitner and Cito \(2016\)](#) characterise performance variation across public infrastructure providers. Their concern is variance between nominally identical environments. Ours is adjacent and, we think, less examined: a *systematic* difference between environments that are not identical and are routinely treated as though they were — two container base images differing in a bundled library version.

Cryptographic interfaces are hard to use correctly.

The defect reported here is not a cryptographic weakness but a consequence of how a cryptographic interface is invoked, which is a well-documented class. Egele et al. [Egele et al \(2013\)](#) found cryptographic misuse widespread in deployed applications and Georgiev et al. [Georgiev et al \(2012\)](#) found certificate validation broken across a wide range of non-browser software; Nadi et al. [Nadi et al \(2016\)](#) attribute much of it to the interfaces themselves rather than to developer inattention, a position Green and Smith [Green and Smith \(2016\)](#) put directly and Indela et al. [Indela et al \(2016\)](#) respond to with interface proposals; and Acar et al. [Acar et al \(2017\)](#) show experimentally that API design materially changes whether developers produce correct code. Our finding sits within that line with one difference: the interface here admits a use that is *correct*, and costly only for a reason invisible at the call site.

Mining public repositories.

Section 7.5 draws on code hosted publicly, and the hazards of doing so are catalogued by Kalliamvakou et al. [Kalliamvakou et al \(2014\)](#) — among them that many repositories are not software projects, that activity is not uniformly distributed, and that a search result is not a random sample. Our sampling frame is stated in Section 4.5 with those hazards in view, which is why no population proportion is computed from it.

Corrections and replication.

This paper is partly a correction of our own earlier work. Gómez et al. [Gómez et al \(2014\)](#) classify replications in software engineering by what they vary; in

those terms the measurements here are *internal* replications that changed the instrument rather than the protocol, and they overturned the earlier conclusion. We report the correction as a result rather than as an erratum for that reason.

The cost of exceptions.

That exceptions are expensive in managed runtimes, and that their cost is concentrated in construction rather than in unwinding, is long established: Ogasawara et al. [Ogasawara et al \(2006\)](#) optimise around exception paths in a Java virtual machine, and Renwick et al. [Renwick et al \(2019\)](#) quantify the cost in a compiled setting. Parravicini and Mueller [Parravicini and Mueller \(2021\)](#) provide the closest available cost-attribution baseline for the V8 engine, though on speculation rather than exceptions. None measures what our result turns on, which we state below.

Key-type confusion.

That a token declaring an HMAC algorithm must not be validated against asymmetric key material is long established; RFC 8725 [Sheffer et al \(2020\)](#) describes the attack and its mitigations. Section 8 reports no new attack. It reports that an existing defence depends on the control flow whose cost this paper measures, so that removing the cost naively removes the defence.

2.1 What we searched for and did not find

Five areas bear directly on this result and we were unable to locate verifiable work in any of them. We state each as an absence we searched for rather than as a claim about the whole literature; each is also, in part, why the misattribution corrected here survived as long as it did.

OpenSSL 3.x performance regressions.

Section 6 attributes magnitude to a bundled OpenSSL version. We found no peer-reviewed study of performance regressions across the OpenSSL 3.x series. Evidence exists, but as project issues and mailing-list discussion — for instance [openssl/openssl issues #17627](#) and [#20698](#), which concern performance of the 3.x provider and decoder machinery. We cite these as engineering evidence and note explicitly that they are not peer-reviewed and were not produced under a measurement protocol we can assess.

Base image to bundled library to application performance. We found no academic study tracing that chain. Malka et al. [Malka et al \(2026\)](#) address build reproducibility and Cito et al. [Cito et al \(2017\)](#) ecosystem practice; neither measures how the library version an image pins changes application performance.

Cryptographic API misuse as a performance defect. The misuse literature cited above treats misuse exclusively as a security problem. We found no work treating a correct-but-costly use of a cryptographic interface as a latency or throughput defect, which is precisely what this paper reports.

Error construction cost in V8 or Node.js. Our result turns on the cost of constructing and discarding an exception, and stack capture is a plausible component of it. We found no measurement of **Error** construction cost specific to V8 or Node.js, so we have no prior baseline to compare against and do not decompose that cost further here.

Profiling inside a cryptographic library. We found no study that profiles within a cryptographic library to attribute time to key parsing and loading rather than to the primitive. Section 5 does this from outside the library, by

differencing and by frame attribution, which is weaker than instrumenting it directly.

3 Correcting our own earlier work

This paper withdraws two claims we made in an earlier, unpublished manuscript [Katha et al \(2026a\)](#). We state them here, before the methodology, rather than in threats to validity, because what replaced them is the substance of this paper.

Withdrawn: event-loop saturation under synchronous cryptography.

The earlier work argued that layered enforcement drives a single-threaded Node.js service toward event-loop saturation, and proposed relocating verification into the service mesh before that point is reached. The saturation curve it rested on was obtained on a host shared with the load generator, the ingress and the sidecars. When the same configuration was re-measured with a background indexing process consuming host CPU, the collapse point moved by hundreds of requests per second, which is recorded with the run in the replication package [Katha et al \(2026b\)](#). A curve that moves with unrelated host activity does not characterise the service. We withdraw the claim and make no capacity claim in this paper.

Withdrawn: cost attributed to key conversion.

A later revision established that the signature primitive is a small share of per-request cost — which survives — and attributed the remainder to per-call conversion of key material. Section 5 measures that conversion at $0.583 \mu\text{s}$. The

attribution was wrong by roughly the ratio between that figure and $18.04 \mu\text{s}$. The same revision explained an asymmetry between HMAC and RSA costs as conversion being dearer than parsing a PEM; a successful asymmetric parse costs $13.83 \mu\text{s}$ against $0.583 \mu\text{s}$ for the conversion, so that explanation is backwards. The real asymmetry is that the RSA path’s parse *succeeds*, making its cost useful work, while the HMAC path’s fails, making it waste.

What the two errors share.

In each case a quantity was attributed to the variable of interest while a second variable moved alongside it, unrecorded at the granularity that would have revealed it: host load in the first, the bundled OpenSSL version in the second. That is the shape Mytkowicz et al. [Mytkowicz et al \(2009\)](#) describe, and it is why this paper records the identity of the component under measurement on every row of every measurement file rather than once per run.

4 Methodology

4.1 Unit of measurement

The unit is one operating-system process, not one call. Each process performs 40000 timed iterations of a single condition after a warmup of half that number and contributes its median. Reported figures are medians of per-process medians, with percentile bootstrap intervals taken over processes.

An interval computed over the calls inside one process describes that process’s steady state and is far too narrow: it cannot observe the compilation decisions, inline-cache states and heap layouts that differ between runs of the

same code [Georges et al \(2007\)](#). Conditions in the decomposition of Section 5 were run in 15 separate processes of 40000 calls; conditions in the runtime matrix of Section 6 in 10 processes of 20000 calls. The largest between-process coefficient of variation is 3.5% on the host; in the matrix it is 20.2%, occurring on the `probe_succeeds` condition under `node:26.6.0-bookworm` rather than across the matrix generally.

Conditions are interleaved within each round rather than run in blocks. Under block ordering a thermal excursion or a background process lands entirely on whichever condition is executing and is indistinguishable from an effect; interleaved, it is distributed across conditions and appears as between-round variance. The clock’s own cost is measured in the same process as the condition it accompanies and is reported rather than subtracted: 0.042 μ s against the smallest quantity reported, 0.583 μ s.

4.2 Decomposing the call

`jwt.verify()` is decomposed into the operations it performs, each timed in isolation under the same protocol: structural decode; the HMAC primitive with a string key and with a pre-parsed key; the symmetric key constructor; and the asymmetric key parse the library attempts first, in both the case where it throws and the case where it succeeds.

4.3 Separating the runtime from the deployment

The earlier manuscript compared a host microbenchmark against an in-container measurement and attributed the difference to deployment. Those environments differed in more than deployment, so the comparison could not support the

attribution. Here containerisation is held constant and the runtime is varied: four container images spanning two OpenSSL series and three V8 major versions, on one machine under one container engine.

The decisive pair is Node 18 and Node 20: they ship V8 10 and V8 11 respectively while sharing the OpenSSL 3.0 series. In stock images Node and OpenSSL versions otherwise co-vary, which would leave the attribution under-determined. Because that pairing carries the attribution, the runtime identity of each image — Node, OpenSSL and V8 version, libc, architecture and image digest — is recorded as a datum and cross-checked against the versions each measurement row carried. That check found 0 mismatches across the matrix.

4.4 In-situ measurement

The deployed service exposes a histogram timing the verification call alone. The instrument is a *paired-scrape difference*: the cumulative histogram is read before and after each arrival-rate step, and the two readings are differenced, which isolates that step’s distribution from the cumulative counters. Means are computed as the difference of the histogram’s `_sum` divided by the difference of its `_count`, so they are exact and do not depend on bucket boundaries; bucket resolution bounds the reported percentiles only.

Seven arrival rates are driven, and the whole sweep is run twice on one process: once in ascending order and once in descending order. We report the *range across the two passes*. This is a range of two, not a confidence interval; no standard error is computed from it, and it is not written as a symmetric tolerance. The opposite-order design also serves as a drift test, which Section 7.2 reports.

4.5 Prevalence: three instruments, three frames

Prevalence is approached with three instruments that fail in different ways. Their sampling frames differ and are not interchangeable, so no population prevalence is computed from them.

Population co-occurrence counts. Code-search counts over public repositories, in four cells: two languages by two import forms. The denominator of a cell is files importing the library; the numerator is files that *also mention* the pre-parsing interface. This measures co-occurrence of two strings in one file, not verified usage.

A classified sample of call sites. Call sites located by search, deduplicated to one file per repository so that no project dominates, with the key argument traced within its file. This is a convenience sample from a relevance-ranked, undocumented ranking; inclusion probabilities are unknown and no weighting back to the population is possible.

A targeted counter-search. Every file co-occurring the library with the pre-parsing interface, fetched in full and adjudicated by hand. This frame is deliberately biased toward finding the pattern — that is its purpose as a negative control — and it is therefore disqualified as a prevalence estimator.

Three reasons preclude a population-level prevalence. The population numerator and the verified numerator measure different things: of 13 automatically flagged call sites, 5 were false positives on hand inspection, so the co-occurrence signal carries an error rate that would be inherited, unmeasured, by any extrapolation. The sample has no inclusion probabilities. And the counter-search is selected on the outcome. Results are therefore reported *in our sample*,

Table 1 Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes.

Operation	Median (μ s)	95% CI
verify(), string secret	29.92	[29.75, 30.12]
failed asymmetric parse	18.04	[17.83, 18.25]
successful asymmetric parse	13.83	[13.62, 13.92]
verify(), pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

or among repositories that co-occur the interface and call the validator, and never as a population proportion.

4.6 Host state is recorded per measurement

The first withdrawn claim in Section 3 failed because host load moved unrecorded. Every in-situ window therefore carries the host load average observed during it, committed beside the measurement, and Section 7.3 uses it.

5 RQ1: Where the cost goes

5.1 The decomposition

Table 1 gives each operation in the validation path, timed in isolation on the host under the protocol of Section 4.

The two emphasised rows are the result. The signature primitive costs 2.208 μ s of a 29.92 μ s call. The conversion to which the remainder has been attributed, including by us, costs 0.583 μ s. The failure that precedes that conversion costs 18.04 μ s, which is 60% of the whole call and its largest single component.

Supplying a pre-parsed key instead of a string reduces the call by 23.92 μ s [23.58, 24.00] over 15 paired rounds.

5.2 The parts do not account for all of it

If the penalty were exactly the discarded failure plus the conversion, the two would sum to it. They do not, quite:

observed penalty	23.92 μ s
failed parse + conversion	18.62 μ s
residual	5.292 μ s

The residual’s interval, [4.916, 5.501], excludes zero. The two measured parts therefore account for 77.9% and not more; the remainder is the library’s own type and claim checking along the non-pre-parsed path. We report it rather than absorbing it into the headline, because a decomposition whose parts are declared to sum exactly to the whole has usually not been checked.

5.3 Confirmation without differencing

The decomposition reaches its conclusion by subtracting conditions — the same inference style this paper criticises elsewhere. It is therefore checked against V8’s sampling profiler, which attributes self time to named frames with no arithmetic on our part. At 50000 iterations per run and 3 runs per environment, matched so the percentages are comparable, the asymmetric key parse accounts for a median 66.2% of self time on the host (range 65.3–67.4 across runs), 95.3% (95.1–95.6) under Node 18, and 61.5% (58.5–61.6) under Node 26. The frame does not appear among the top frames of the pre-parsed path in any environment.

Two methods, one answer. We rely on the ordering and the dominance rather than on the exact percentages: sampling

Table 2 The same measurement across four container runtimes on one machine, 10 processes each of 20000 calls. The host row is from Table 1 and is not containerised; its OpenSSL version is marked n/r because that run predates the per-measurement OpenSSL field and the host’s OpenSSL has since changed, so no version can be recorded for it after the fact (Section 11).

Runtime	OpenSSL	V8	Failed parse (μ s)	Pre-parsed <code>verify()</code>
Node 18, musl	3.0.16	10	373.79	7.58
Node 20, musl	3.0.19	11	420.62	7.25
Node 26, musl	3.5.7	14	22.94	6.56
Node 26, glibc	3.5.7	14	16.58	6.04
host (no container)	n/r	14	18.04	6.042

attribution depends on rate and on what else the process is doing.

6 RQ2: What governs the magnitude

Table 2 varies the runtime with containerisation held constant.

It is OpenSSL, not the JavaScript engine.

Node 18 and Node 20 ship V8 10 and 11 — different major versions of the engine — and share the OpenSSL 3.0 series. Both pay of order 373.79–420.62 μ s. Node 26 ships OpenSSL 3.5 and pays 22.94 μ s on the same libc. Had the Node 20 runtime been fast, the attribution would have been the engine rather than the library; it was not.

The effect is confined to key parsing.

Across the four container rows, pre-parsed validation spans 1.26 $\times$, the HMAC primitive 1.39 $\times$ and structural decode 1.35 $\times$. The failed parse spans 25.36 $\times$. The successful parse moves with it (11.74 $\times$), so this is the cost of OpenSSL 3.0 key parsing in general rather than anything peculiar to the failure path.

The failure is simply the case in which an application pays that cost on every request for a result it discards.

What this design does not isolate.

A reader may notice that the uncontainerised host row is of the same order as the Node 26 container row. That comparison is confounded and we draw no conclusion about containerisation from it: the host runs a different operating system from the containers, so operating system, C library and Node build all change alongside the container boundary. Isolating containerisation would require a Linux-native baseline, which we did not obtain. The claims above concern only the four container rows, among which containerisation is constant.

7 RQ3: Deployment and prevalence

7.1 In-situ cost across arrival rates

The deployed service was driven at seven arrival rates, each measured by the paired-scraper difference of Section 4.4, with the whole sweep run ascending and then descending on one process. Table 3 reports the range across the two passes at each rate for the configuration the earlier study deployed, a string secret.

Two observations follow, both visible in Figure 1. The cost is present in deployment and is far larger than the isolated host figure of Table 1, consistent with Section 6: the deployed runtime is the one carrying the older OpenSSL. And the cost depends on arrival rate, falling as the rate rises.

Table 3 In-situ validation cost with a string secret, by arrival rate. The two columns are the ascending and the descending pass over the same seven rates on one process; each is a single window, and the pair is two observations rather than an interval. Host load average during each window is committed with the measurement.

Arrival rate (requests/s)	Mean per call (μ s)	
	ascending	descending
5	928.24	1027.97
10	976.28	1017.34
25	918.54	934.47
50	758.57	737.12
100	571.59	607.78
200	494.91	532.50
400	452.82	467.62

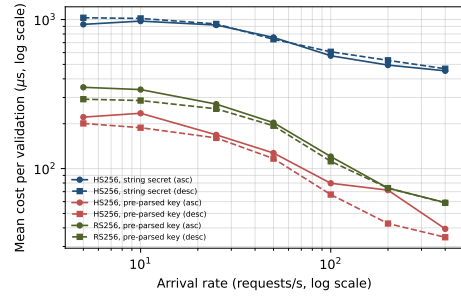


Fig. 1 In-situ validation cost against arrival rate, for three key-form and algorithm combinations, on logarithmic axes. Ascending and descending passes are drawn separately rather than averaged: Section 7.2 reports that they differ systematically, and averaging would conceal that. Every point is read from a committed run directory by `figures/fig_insitu_rate.py` [Katha et al \(2026b\)](#).

7.2 The two passes differ systematically

Sweeping up and then down is not only replication; it detects warm-up, thermal and hysteresis effects. If the difference between passes were noise, its sign would alternate across rate points. It does not. In the string-secret sweep the descending

pass is slower at 6 of 7 rate points; in the pre-parsed sweep the descending pass is *faster* at 7 of 7. A consistent sign across seven points is not what noise produces.

This has a direct consequence for how the ranges in Table 3 should be read. A systematic component means the two passes are not independent samples of a common value, so the range between them understates uncertainty further than a range of two already does. We therefore report ranges and draw no inference that depends on their width. It is also a methodological finding in its own right: a single-direction sweep would have produced a curve carrying a consistent bias with no way to detect it.

7.3 Host load, not the condition, drives the outliers

Agreement between passes varies considerably across windows. We distinguish *relative* disagreement, the gap between passes as a fraction of their mean, from *absolute* disagreement, the gap in microseconds; the two behave differently and the distinction carries this subsection.

The widest *relative* disagreement in the dataset and the tightest occur at the same arrival rate in different sweeps. The wide one is the highest-load window in its own sweep on the descending pass, though not on the ascending pass, whose maximum load falls at a different rate. The tight one, at the same arrival rate in another sweep, sits at a lower load on both passes and its two passes agree to within $0.02\ \mu\text{s}$. Relative disagreement therefore does not track the condition under measurement.

We considered two alternative explanations for the relative asymmetry and reject both. A small-mean effect would

require *absolute* disagreement to be roughly constant, so that its relative size grew only as means shrank; absolute disagreement instead varies over three orders of magnitude and is largest where the means are largest, which is the opposite pattern. A measurement floor would require the affected values to sit at the instrument’s resolution; they do not, and in any case the means reported here are computed from the histogram’s sum and count rather than from its buckets, so bucket resolution cannot bound them.

The operative consequence is that in-situ figures are reported with the host load recorded during their window, and comparisons between windows at different loads are not treated as comparisons between conditions.

7.4 Corroboration: the cost is real processor time

That the in-situ cost is processor time rather than queueing or scheduling delay is corroborated by an A/B of the same service with validation enabled and disabled. Table 4 gives it here rather than leaving it in the replication package Katha et al (2026b), because the claim it carries is load-bearing for everything after it.

Enabling validation adds 76.92 millicores of application CPU at 200.006 requests per second, which is $384.6\ \mu\text{s}$ of processor time per call. That is the corroborating figure: it is arrived at from cgroup CPU accounting, independently of the histogram, and it is the same order as the in-situ means of Table 3. Queueing and scheduling delay are not what moved. Event-loop lag is unchanged between the arms, and the client-side mean rises by $353.0\ \mu\text{s}$, close to the CPU figure rather

Table 4 Validation enabled against validation disabled, at one arrival rate. Two 180 s windows on the same pod running the same image digest, switched by an environment variable, with 36001 requests in each. A signed token is on the wire in both arms, so what the disabled arm removes is the handler’s `jwt.verify()` call and the header reads around it, not the token. Application and sidecar CPU are means of the one-second cgroup samples falling inside the window; per-call CPU is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99 and is not a p99 over the window. Host load is the load average sampled at window start; these two runs carry no within-window load series, so no window mean of it exists to report.

	disabled	enabled
Achieved rate (requests/s)	200.006	200.006
Application CPU (m)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (m)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean (ms)	1.6802	2.0332
Client p99 (ms)	6.3480	8.0890
Host load, 1 min	3.53	5.11
Host load, 5 min	3.50	4.10
Host load, 15 min	3.18	3.49

than in excess of it, which is what synchronous work on an unsaturated event loop produces.

The two windows are not at equal host load, and the rule stated above bars us from comparing them on wall-clock. It does not bar the CPU comparison, which is what the claim rests on: host contention inflates elapsed time, whereas the application container’s own cgroup accounting measures the processor time it was charged. Three further observations weigh against reading the CPU difference as an artefact of the windows. The sidecar, in the same pod under the same contention, did not rise with it: 53.40 millicores against 49.13. The per-second samples of the two arms do not overlap at all — disabled never exceeded 100.577 millicores in 127 samples, enabled

never fell below 114.963 in 127 — which separates the arms without assuming anything about the distribution, as a test over autocorrelated one-second samples would have to. And the two host-load indicators disagree in sign: the load average was higher entering the enabled window, while the busiest process on the host at that instant was lower there than in the disabled window, 84.3% against 135.1%. The enabled window began 78 s after the disabled window ended, and a one-minute load average decays over about that interval, which is the likeliest reading of a gap that narrows from the one-minute to the fifteen-minute figure.

What the A/B does not do is attribute the cost within validation. It brackets the whole `jwt.verify()` call; Section 5 is what divides it.

7.5 How common is the avoiding pattern?

Three instruments, three frames, reported separately as Section 4.5 requires.

Population co-occurrence.

Of 790528 public JavaScript files importing the library through `require`, 29 also mention the pre-parsing interface; for JavaScript through `import` the figures are 524288 and 38; for TypeScript through `require`, 8288 and 0; and for TypeScript through `import`, 333824 and 101. Cells are not pooled into a rate: a file using both import forms would be counted twice, so their sum 1656928 is an upper bound on the union and the largest cell 790528 a lower bound. No cell exceeds 0.0303% of files mentioning the interface. These are search-engine estimates of file counts captured on 2026-09-18, they cover public repositories only, and they measure co-occurrence rather than use.

The denominators are returned *rounded* — each is an exact multiple of 1024 — so they should be read as magnitudes rather than as counts we established file by file.

Classified call sites.

Of 327 call sites classified across 247 distinct repositories, 0 pass a pre-parsed key. Environment variables account for 176 of those call sites, untraced identifiers and member expressions 93, string literals 52, file contents 4 and buffers 2. No file in this corpus contains the pre-parsing interface at all, in any spelling, so the zero reflects the pattern’s absence from the sample rather than a classifier that cannot see it. Incidentally, 309 of 327 call sites (94%) pass no algorithm restriction, which Section 8 returns to.

An earlier version of the classifier recognised only the conventional `jwt.verify()` spelling, which filtered the corpus toward one import idiom. It was re-run over the previously unclassified residue with a matcher that resolves the local binding of the import — covering arbitrary aliases, namespace imports, destructured and renamed imports, and direct calls on the `require` expression. That recovered 12 further call sites, included in the totals above, of which 0 pass a pre-parsed key.

Targeted counter-search.

A sample can only show a pattern is rare if it could have found it. Every file co-occurring the library with the pre-parsing interface was therefore fetched: 191 files, of which 120 contain a validation call, spanning 112 distinct repositories. An automated pass flagged 13 call sites as passing a pre-parsed key and all were then read by hand: 5 are false positives — projects defining their own helper function of the same name, one of which

returns a string — 1 could not be resolved within its file, and 7 are genuine, across 6 distinct projects.

The rate, with its denominator.

Among the 112 repositories that both reference the pre-parsing interface and call the validator, 6 use it: 5.36%. That denominator is the most favourable available, since those projects demonstrably know the interface exists, and it is not the population. The pattern is rare rather than absent, and it is rare even among projects already aware of it.

8 A fix, and a constraint on it

The fix is to dispatch on the token’s declared algorithm before attempting the asymmetric parse, so that an HMAC token with an ordinary string secret is resolved directly. Applied, this takes the call from 29.92 μ s to 7.208 μ s, a factor of 4.15, and to within 1.208 μ s of supplying a pre-parsed key, with no change in calling code. That residual is the median of the per-round paired differences, not the difference of the two medians; the two are not identical and the former is reported throughout.

We tested it against the library’s own suite rather than only against assertions of our own. Cloned at the release tag, the patch applies cleanly and the suite is unchanged: 511 passing and 0 failing unpatched, 511 and 0 patched.

The constraint, which is the more important half.

The discarded attempt is not purely wasteful. The *success* of that parse on asymmetric key material is what produces a key object whose type a later check depends on, and that check is

what prevents symmetric and asymmetric material from being interchanged [Sheffer et al \(2020\)](#). A change that dispatches on the declared algorithm while relaxing which key material may take the fast path alters which inputs reach that check.

The fix must therefore be narrow: the fast path is restricted to string secrets that cannot be asymmetric key material, so anything that might be resolves exactly as it does today. Removing that material restriction — the form of the optimisation we argue against — produces 2 failures in the library’s own suite, both in its key-confusion tests.

A reduction of our own contribution.

We had written a regression test for this property and intended to offer it upstream. Running the library’s existing suite showed that it already constrains the change: a contributor proposing the unrestricted optimisation would be caught by tests they are required to run. We therefore withdraw that contribution. What remains is narrower and, we think, still worth stating: the cost documented in Section 5 cannot be removed by the obvious route, and the reason is a property the maintainers already test for but which is not evident from the code being changed.

The independent defence against this class of confusion is for the caller to restrict permitted algorithms [Sheffer et al \(2020\)](#), and 94% of the call sites surveyed in Section 7.5 do not.

9 Offloading is not the remedy

Because a cost believed to be cryptographic invites relocation, we implemented the relocation the earlier work

proposed and report what it does. This section rests on source inspection and a manual observation; it carries no measured result, for the reason given below.

The mechanism, which a reader can verify from the source.

Applying a mesh `RequestAuthentication` policy causes the sidecar to validate the token before proxying. It does not cause the application to stop validating: verification is *duplicated*, not relocated, and no work leaves the event loop. Making relocation effective therefore requires the application to trust something the sidecar tells it. In our implementation the handler reads a sidecar-populated claims header and, when it is present, returns the protected payload without verifying anything — the branch is a single conditional in `service-a/index.js` in the replication package [Katha et al \(2026b\)](#). A malformed header returns a server error rather than falling back to local verification, which is the fail-closed behaviour that branch intends.

The observation.

That trust is the vulnerability. The sidecar overwrites the header only while a policy is applied; in any other state nothing sanitises it. With the guard filter applied, a request carrying a forged claims header and no credential at all is refused. With the guard removed, the same request returns the protected payload. We reproduced this sequence manually — refused, served, refused — and it is consistent with the note recorded alongside the affected runs in the replication package. The remedy is an always-applied filter that strips any client-supplied copy of the header on entry to the pod, before any validation filter runs, matched on

the connection manager rather than on the validation filter, because in the state that needs protection no validation filter exists.

Why we report no measured result.

We wrote an automated probe to turn the observation into a table and it does not produce a trustworthy one. The mesh applies a configuration change asynchronously, on a timescale comparable to the probe interval, so whether a probe observes the guarded or the unguarded behaviour depends on whether the proxy had converged when the request fired. Two runs of the same script produced contradicting values for the same configuration. Rather than report the run that agreed with our expectation, we report none: the script's own negative control declared both runs inconclusive, and we retain it in the replication package with that limitation recorded. Making this measurable requires gating each probe on *observed* configuration convergence — polling until repeated identical responses — rather than on a fixed delay. We have not implemented that, and until it is done this section should be read as a mechanism argument supported by an observation, not as a measurement.

10 Implications for practice

Pre-parse key material once, at startup.

On the host this removes 23.92 μ s per validation [23.58, 24.00]; on the runtime the service actually deployed, where the discarded parse costs 373.79 μ s, it removes far more. It requires no architectural change, no new component and no movement of a trust boundary.

Treat the base image as part of the measurement.

The same code, the same library and the same architecture differ by 25.36 $\times$ on this path across the container images measured, because they pin different OpenSSL versions. A performance figure quoted without the base image it was obtained on is not reproducible, and a regression appearing after a base-image bump may have nothing to do with the application.

Pin the algorithm.

Independently of anything here, callers should restrict permitted algorithms [Sheffer et al \(2020\)](#). Most of the call sites we surveyed do not, which leaves them relying entirely on library-internal key typing for a property they could assert directly in one argument.

Do not reach for an architectural remedy first.

Section 9 is the argument against treating relocation as the default response: the cost is not cryptographic, it is largely avoidable in place, and the remedy that moves validation elsewhere also moves a trust boundary.

11 Threats to validity

One architecture.

Every measurement comes from a single ARM64 host and its container runtimes under one engine. We have not verified any of this on x86_64. The mechanism is a property of a library's API contract — an asymmetric parse attempted before a symmetric one, with the failure discarded — and of the cost of key parsing in a particular OpenSSL series, and neither is a property of an instruction set. But an expectation is not a measurement, and

the magnitude in particular depends on how OpenSSL’s key parsing performs on a given platform, which is exactly the kind of quantity that varies with architecture. No claim here should be read as holding beyond ARM64.

Ranges of two, with a systematic component.

The in-situ dispersion is the range across an ascending and a descending pass. It is not a confidence interval, and Section 7.2 shows the two passes differ systematically rather than randomly, so they are not independent samples of a common value and the range understates uncertainty further. No inference in this paper depends on the width of those ranges.

Sampling frames are not interchangeable.

The three prevalence instruments of Section 4.5 have different units and different selection mechanisms, and none supports a population proportion. The co-occurrence counts measure two strings appearing in one file, with a false-positive rate established at 5 of 13 on hand inspection. The classified sample has unknown inclusion probabilities, a limitation of repository search noted more generally by Kalliamvakou et al. [Kalliamvakou et al \(2014\)](#). The counter-search is selected on the outcome.

The matcher’s coverage.

Call sites are found by resolving the local binding of the import, which covers aliases, namespace imports, destructured and renamed imports and direct calls on the require expression. It does not resolve bindings across files, so a project that constructs a pre-parsed key in one file and validates in another is invisible to this analysis. This is the strongest threat to

Section 7.5; within-file tracing found no such case, which is weak evidence against it.

Host load, and the confound that motivated recording it.

The first claim withdrawn in Section 3 failed because host load moved unrecorded between runs. Every in-situ window here carries the load average observed during it, and Section 7.3 shows that window-to-window agreement tracks that load rather than the condition. Contention inflates a duration rather than deflating it, so in-situ figures are upper bounds; but comparisons between windows at different loads are not comparisons between conditions, and we do not treat them as such.

One host figure cannot be version-attributed.

The host decomposition of Section 5 predates the per-measurement OpenSSL field, and the host’s OpenSSL has since moved by a patch release, so the version that produced those figures is known by run timestamp rather than from the measurement record. Table 2 therefore leaves the host OpenSSL cell unrecorded rather than filling it with a version captured later. The container rows, which carry the attribution of Section 6, each record their own.

Token caching is not in the comparison.

A service that caches verified tokens pays this cost once per token rather than once per request. We did not measure such a configuration, so the per-request figures here do not describe a caching deployment.

One rater.

The 13 automatically flagged call sites were read and classified by the first author, and the authors are not disinterested: the paper’s claim is stronger the more of them are false positives, and 5 were classified that way. There was no second rater and no inter-rater agreement to report. The per-case verdicts, the evidence for each and the hashes of the files they were read from are published so the adjudication can be repeated, which is weaker than having repeated it.

One library at one version.

The mechanism is specific to one JavaScript implementation at one release. Whether other implementations or other language ecosystems attempt key resolution in the same order is unmeasured.

12 Conclusion

The per-request cost of token validation in this library is real, is worth attending to, and is not cryptographic: the signature primitive is $2.208\ \mu\text{s}$ of a $29.92\ \mu\text{s}$ call. It is not key conversion either, which costs $0.583\ \mu\text{s}$. It is a parse attempted in the wrong order, whose failure is constructed and thrown away on every request at $18.04\ \mu\text{s}$, 60% of the call. Its price is set by the OpenSSL version a base image happens to pin, varying $25.36\times$ across runtimes while the rest of the validation path varies by $1.26\times$. The pattern that avoids it is rare: among 112 repositories that already reference the pre-parsing interface and call the validator, 6 use it.

We withdraw two earlier claims in reaching this: that layered enforcement drives the service to event-loop saturation, which rested on a curve that moved with host load; and that the dominant cost is key conversion, which a direct

measurement of the conversion refutes. Both have the same shape, and it is the shape this paper’s methodology is built to avoid: a cost attributed to the variable under study while a second variable moved alongside it, unrecorded at the granularity at which it could change. The methodological lesson is not that one should measure in deployment. It is narrower: record the identity of the component whose behaviour is being measured, on every measurement, at the granularity at which it can change.

Declarations

Data availability. The data supporting the results of this study are openly available in the archived replication package [Katha et al \(2026b\)](#), deposited on Zenodo under the DOI given in that reference. It contains all measurements, the instrumentation, the analysis code and the scripts that generate every numeric value and the figure in this paper. Every number is generated from a file under the run or survey directories; none is entered by hand. The survey corpus is third-party source and is not redistributed; a manifest pinning every file examined by content hash stands in for it, covering 502 files across 457 repositories. Content hashes resolved for all of them; the repository HEAD commit resolved for 501 of 502, one repository having become unavailable between fetch and manifest. The content hash is the authoritative pin and is complete.

Competing interests. The authors declare no competing interests.

Funding. [To be completed by the authors before submission.]

Author contributions. [To be completed by the authors before submission. Springer requires a statement of each author’s contribution; the present authors’ individual roles are not recorded in the artifact and are not inferable from it.]

Ethics approval. Not applicable. This study measures software behaviour and publicly available source code; it involves no human participants and no personal data.

References

- Acar Y, Backes M, Fahl S, et al (2017) Comparing the usability of cryptographic APIs. In: 2017 IEEE Symposium on Security and Privacy (SP), <https://doi.org/10.1109/SP.2017.52>
- Chen J, Shang W (2017) An exploratory study of performance regression introducing code changes. In: 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), <https://doi.org/10.1109/ICSME.2017.13>
- Cito J, Schermann G, Wittern JE, et al (2017) An empirical analysis of the Docker container ecosystem on GitHub. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), <https://doi.org/10.1109/MSR.2017.67>
- Coarfa C, Druschel P, Wallach DS (2006) Performance analysis of TLS web servers. *ACM Transactions on Computer Systems* 24(1). <https://doi.org/10.1145/1124153.1124155>
- Egele M, Brumley D, Fratantonio Y, et al (2013) An empirical study of cryptographic misuse in Android applications. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS ’13), <https://doi.org/10.1145/2508859.2516693>
- Georges A, Buytaert D, Eeckhout L (2007) Statistically rigorous Java performance evaluation. In: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA ’07), <https://doi.org/10.1145/1297027.1297033>
- Georgiev M, Iyengar S, Jana S, et al (2012) The most dangerous code in the world: Validating SSL certificates in non-browser software. In: Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS ’12), <https://doi.org/10.1145/2382196.2382204>
- Gómez OS, Juristo N, Vegas S (2014) Understanding replication of experiments in software engineering: A classification. *Information and Software Technology* 56(8). <https://doi.org/10.1016/j.infsof.2014.04.004>
- Green M, Smith M (2016) Developers are not the enemy! the need for usable security APIs. *IEEE Security & Privacy* 14(5). <https://doi.org/10.1109/MSP.2016.111>
- Indela S, Kulkarni M, Nayak K, et al (2016) Helping Johnny encrypt: Toward semantic interfaces for cryptographic frameworks. In: Proceedings of the 2016 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2016), <https://doi.org/10.1145/2898859.2898860>

- [//doi.org/10.1145/2986012.2986024](https://doi.org/10.1145/2986012.2986024)
- Jones MB, Bradley J, Sakimura N (2015) JSON web token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force, <https://doi.org/10.17487/RFC7519>
- Kalliamvakou E, Gousios G, Blincoe K, et al (2014) The promises and perils of mining GitHub. In: Proceedings of the 11th Working Conference on Mining Software Repositories (MSR 2014), <https://doi.org/10.1145/2597073.2597074>
- Kaltenecker C, Mühlbauer S, Grebhahn A, et al (2023) Performance evolution of configurable software systems: An empirical study. *Empirical Software Engineering* 28(6). <https://doi.org/10.1007/s10664-023-10338-3>
- Katha JF, Prova TT, Morol MK, et al (2026a) Quantifying the performance cost of API security in cloud-native microservices: An empirical study of TLS, mTLS, and JWT validation. Unpublished manuscript by the present authors. Both of its claims discussed here are withdrawn; see Section 3.
- Katha JF, Prova TT, Morol MK, et al (2026b) Replication package: where the cost of JWT verification actually goes. [https://doi.org/\[ZENODOITOBE MINTED BEFORE SUBMISSION\]](https://doi.org/[ZENODOITOBE MINTED BEFORE SUBMISSION]), archived dataset. Development repository: <https://github.com/Katha-Sikdar/microservices-api>
- Laaber C, Leitner P (2018) An evaluation of open-source software microbenchmark suites for continuous performance assessment. In: Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18), <https://doi.org/10.1145/3196398.3196407>
- Laaber C, Scheuner J, Leitner P (2019) Software microbenchmarking in the cloud. how bad is it really? *Empirical Software Engineering* 24(4). <https://doi.org/10.1007/s10664-019-09681-1>
- Leitner P, Cito J (2016) Patterns in the chaos—a study of performance variation and predictability in public IaaS clouds. *ACM Transactions on Internet Technology* 16(3). <https://doi.org/10.1145/2885497>
- Malka J, Zacchiroli S, Zimmermann T (2026) Docker does not guarantee reproducibility. arXiv preprint arXiv:2601.12811, no DOI registered; verified against the arXiv record
- Mytkowicz T, Diwan A, Hauswirth M, et al (2009) Producing wrong data without doing anything obviously wrong! In: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09), <https://doi.org/10.1145/1508244.1508275>
- Nadi S, Krüger S, Mezini M, et al (2016) Jumping through hoops. In: Proceedings of the 38th International Conference on Software Engineering (ICSE '16), <https://doi.org/10.1145/2884781.2884790>
- Naser A, Sadeghi Lahijani M, Wu C, et al (2020) Performance evaluation and modeling of cryptographic libraries for MPI communications. arXiv preprint arXiv:2010.06139, no DOI registered; verified against the arXiv record

- Ogasawara T, Komatsu H, Nakatani T (2006) EDO: Exception-directed optimization in Java. *ACM Transactions on Programming Languages and Systems* 28(1). <https://doi.org/10.1145/1111596.1111598>
- Parravicini A, Mueller T (2021) The cost of speculation: Revisiting overheads in the V8 JavaScript engine. In: 2021 IEEE International Symposium on Workload Characterization (IISWC), <https://doi.org/10.1109/IISWC53511.2021.00013>
- Renwick J, Spink T, Franke B (2019) Low-cost deterministic C++ exceptions for embedded systems. In: Proceedings of the 28th International Conference on Compiler Construction (CC 2019), <https://doi.org/10.1145/3302516.3307346>
- Sheffer Y, Hardt D, Jones MB (2020) JSON web token best current practices. Request for Comments RFC 8725, Internet Engineering Task Force, <https://doi.org/10.17487/RFC8725>
- Sikeridis D, Kampanakis P, Devetsikiotis M (2020) Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In: Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '20), <https://doi.org/10.1145/3386367.3431305>
- Traini L, Cortellessa V, Di Pompeo D, et al (2023) Towards effective assessment of steady state performance in Java software: Are we there yet? *Empirical Software Engineering* 28(1). <https://doi.org/10.1007/s10664-022-10247-x>
- Tuveri N, Brumley BB (2019) Start your ENGINES: Dynamically loadable contemporary crypto. In: 2019 IEEE Cybersecurity Development (SecDev), <https://doi.org/10.1109/SecDev.2019.00014>
- Zhu X, She G, Xue B, et al (2023) Dissecting overheads of service mesh sidecars. In: Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC '23), <https://doi.org/10.1145/3620678.3624652>